

Tag 1 - Wenn ein Bit kippt: Prüfziffern und erste Fehlererkennung

Reise-Ziel der zwei Wochen: Verstehen, wie ein Datamatrix-Code Fehler verkraftet (zerkratzte Pakete, dreckige Etiketten, schlechte Drucker). Heute bauen wir das Fundament: **Wie erkennt man überhaupt, dass etwas schief gelaufen ist?**

Lernziele für heute

Am Ende von Tag 1 kannst du ...

- erklären, warum Computer mehr Daten speichern, als sie eigentlich müssen
- mit der Modulo-Rechnung umgehen
- ein Paritätsbit von Hand und in Python berechnen
- die Prüfziffer eines EAN-13-Strichcodes nachrechnen
- konkret sagen, **welche Fehler ein Verfahren erkennt - und welche nicht**

Material: Bleistift, Radiergummi, kariertes Papier, ein Laptop mit Python, ein Produkt mit Strichcode (Schokolade, Buch, Müslipackung – egal).

Block 1 · Warum überhaupt Fehlerkorrektur? (≈ 20 min)

Stell dir vor, du scannst an der Supermarktkasse eine Schokolade. Der Strichcode ist zerkratzt, die Kassierer*in hört kein „Piep“. Was tun?

1. **Erkennen**, dass der Scan kaputt ist → nochmal scannen.
2. **Korrigieren**: Der Scanner rät die fehlende Stelle richtig → kein zweiter Scan nötig.

Beides braucht **Redundanz**: Wir speichern absichtlich mehr, als nötig wäre. Der „Mehraufwand“ ist der Preis dafür, dass Daten auch dann noch ankommen, wenn unterwegs etwas verloren geht.

Mini-Aufgabe (mündlich, 5 min)

Sammelt zusammen 3 Situationen aus deinem Alltag, in denen Fehler in Daten passieren können. (Tipps: SMS, WLAN, CD-Player, USB-Stick, QR-Code im Bus, Banküberweisung ...)

Die naivste Idee: Dreimal schicken

Wir senden jedes Bit **dreimal**:

Original: 1 0 1 1
Gesendet: 111 000 111 111
Empfangen: 111 010 111 111 ← in der Mitte ist 1 Bit gekippt

Der Empfänger schaut bei jedem Dreierblock: „Welches Bit ist häufiger?“ → Mehrheitsentscheid.



Bleistift-
Übung 1

Wiederholung als Code

1. Welche der folgenden empfangenen Dreierblöcke werden korrekt zu 0 oder 1 zurückübersetzt? 111, 010, 001, 110, 000, 101
2. Wie viele Bitfehler pro Dreierblock kann das Verfahren **korrigieren**? Wie viele kann es nur **erkennen**?
3. Wie viel Prozent der gesendeten Bits sind „Verschwendung“ (Redundanz)?
4. Diskutiert: Warum verwendet niemand dieses Verfahren in der Praxis?

→ Wir wollen klüger sein als „dreimal schicken“. Auf gehts.

Block 2 · Mathe-Werkzeug: Modulo (≈ 20 min)

Fast jedes Fehlerkorrektur-Verfahren rechnet mit **Modulo**. Es lohnt sich, die fünf Minuten zu investieren.

$a \bmod n$ heißt: „Was bleibt als Rest, wenn ich a durch n teile?“

$17 \bmod 5 = 2$ (denn $17 = 3 \cdot 5 + 2$)
 $23 \bmod 7 = 2$
 $100 \bmod 13 = 9$

Bild: Eine Uhr ist Modulo 12. Wenn es 10 Uhr ist und du 5 Stunden wartest, ist es nicht 15 Uhr, sondern 3 Uhr. Also $(10 + 5) \bmod 12 = 3$.

In Python ist Modulo der %-Operator:

```
>>> 17 % 5  
2  
>>> (10 + 5) % 12  
3
```



Modulo warmrechnen

Berechne im Kopf oder auf Papier:

Bleistift-
Übung 2

Aufgabe	Ergebnis
29 mod 7	
100 mod 9	
1000 mod 11	
(47 + 38) mod 10	
(8 · 7) mod 10	
0 mod 13	

Wichtige Eigenschaft, die wir oft brauchen:

$$(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$$

Heißt: Du darfst beim Rechnen jederzeit „mod n“ nehmen, ohne dass das Ergebnis falsch wird. Das ist Gold wert, wenn die Zahlen groß werden.

Block 3 · Das Paritätsbit - die einfachste Erkennung (≈ 45 min)

Idee: Wir hängen an unsere Daten **ein einziges Bit** an, und zwar so, dass die Anzahl der Einsen am Ende **gerade** ist. Das nennt man *gerade Parität*.

Beispiel: Daten = 1011010 → drei Einsen, also ungerade → wir hängen eine 1 an → 10110101.

Beim Empfänger: Anzahl der Einsen zählen. Ist sie ungerade → Fehler erkannt.



Parität von Hand

Bleistift-
Übung 3

(a) Bestimme das Paritätsbit (gerade Parität) für:

Datenwort	Paritätsbit	Gesendetes 8-Bit-Wort
1010101		
1111000		
0000001		
1100110		
0000000		

(b) Welche dieser empfangenen 8-Bit-Wörter sind **mit Sicherheit fehlerhaft** (gerade Parität vorausgesetzt)?

10110011
11000001
00000000
01010100
11111111

- (c) **Wichtigste Frage des Tages:** Was passiert, wenn unterwegs **zwei** Bits kippen? Erkennt das Paritätsbit den Fehler? Begründe.
- (d) Die Parität erkennt also nur eine bestimmte Sorte Fehler. Beschreibe in einem Satz, welche.



Werkzeug-
Check

Thonny

Bevor wir den ersten Code schreiben, kurz das Werkzeug einrichten. Wir nehmen **Thonny** – eine Python-Umgebung, die genau für Einsteiger gemacht ist. Falls noch nicht installiert: thonny.org → herunterladen → starten.

So sieht Thonny aus:

- Oben der **Editor** – dort schreibst du dein Programm (mehrere Zeilen, wird gespeichert).
- Unten die **Shell** – dort kannst du einzelne Befehle direkt ausprobieren, sie werden sofort ausgeführt. Perfekt zum Herumspielen.

Die fünf Befehle, die du heute brauchst:

Aktion	Tastenkombination	Zweck
Neue Datei	Strg + N	Frisches leeres Editor-Fenster
Speichern	Strg + S	Code als .py-Datei sichern
Programm ausführen	F5	Den Editor-Code starten
Eine Zeile testen	direkt in Shell tippen + Enter	Schnellprobe ohne Speichern
Variable anschauen	Menü „Ansicht“ → „Variablen“	Zeigt, was gerade gespeichert ist

Erste Probe (in der Shell):

```
>>> 2 + 3
5
>>> 17 % 5
2
>>> print("Hallo Greta")
Hallo Greta
```

Tipp - der Debugger: Thonny hat einen wunderbaren Debug-Knopf (das Käfer-Symbol). Damit lässt sich Code Schritt für Schritt ausführen und du siehst, was bei jedem Schritt passiert. Heute brauchst du das noch nicht, aber merken!

Wenn etwas nicht funktioniert: Lies die rote Fehlermeldung ganz unten. Sie zeigt meistens **die Zeile** und **das Problem**. Häufige Stolperer: vergessenes `:` am Ende von `for/if`-Zeilen, falsche Einrückung (Python ist da streng!), Tippfehler in Variablenamen.

Python-
Einheit 1

Paritätsprüfer bauen

Lege in Thonny eine neue Datei an (Strg + N), speichere sie als `paritaet.py`, tippe den Code unten ab und drücke F5 zum Ausführen:

```
def paritaetsbit(daten):  
    """Liefert 0 oder 1, so dass die Gesamtanzahl der Einsen gerade ist."""  
    return sum(daten) % 2  
  
def ist_gueltig(wort):  
    """Prüft, ob ein Wort gerade Parität hat."""  
    return sum(wort) % 2 == 0  
  
# Test:  
daten = [1, 0, 1, 1, 0, 1, 0]  
p = paritaetsbit(daten)  
gesendet = daten + [p]  
print("Gesendet:", gesendet, " gültig?", ist_gueltig(gesendet))  
  
# Jetzt einen Fehler einbauen:  
empfangen = gesendet.copy()  
empfangen[3] = 1 - empfangen[3] # Bit an Position 3 kippen  
print("Empfangen:", empfangen, " gültig?", ist_gueltig(empfangen))
```

Aufgabe 3.1 - Systematischer Test Schreibe Code, der **jedes** mögliche 7-Bit-Datenwort durchgeht (das sind 128), je ein Paritätsbit anhängt, dann **genau einen Bitfehler** an jeder möglichen Position simuliert und prüft, ob die Parität den Fehler erkennt.

Erwartetes Ergebnis: 100 % aller Einzelfehler werden erkannt.

```
def kippe_bit(wort, position):  
    neu = wort.copy()  
    neu[position] = 1 - neu[position]  
    return neu  
  
# TODO: Greta füllt aus  
# - Schleife über alle 128 möglichen 7-Bit-Datenwörter  
# - Paritätsbit anhängen  
# - Schleife über alle 8 Bitpositionen  
# - Bit kippen, Parität prüfen, mitzählen
```

Tipp: Alle 128 Datenwörter bekommst du mit `itertools.product([0,1], repeat=7)`.

Aufgabe 3.2 - Wo versagt das Verfahren? Mache das Gleiche, aber kippe **zwei** Bits gleichzeitig (an verschiedenen Positionen). Welcher Anteil der Doppelfehler wird erkannt?

Block 4 · EAN-13 - Prüfwziffern aus dem echten Leben (≈ 60 min)

Jetzt verlassen wir die 0/1-Welt und gehen ins echte Leben. Hol dir bitte ein **Produkt mit Strichcode** vom Schreibtisch oder aus der Küche.

Unten am Strichcode stehen 13 Ziffern. Das ist der **EAN-13** (European Article Number). Er steckt auf fast jedem Konsumgut der Welt.

4 0 0 6 3 8 1 3 3 3 9 3 1
 └──────── 12 Datenziffern ─────────┘ |
 Prüfwziffer

Wie wird die Prüfwziffer berechnet?

1. Multipliziere die ersten 12 Ziffern abwechselnd mit **1** und **3** (1. Ziffer × 1, 2. Ziffer × 3, 3. Ziffer × 1, ...).
2. Bilde die Summe.
3. **Prüfwziffer = (10 – (Summe mod 10)) mod 10**

Beispiel: 5 9 0 1 2 3 4 1 2 3 4 5 ?

$$\begin{aligned}
 &5 \cdot 1 + 9 \cdot 3 + 0 \cdot 1 + 1 \cdot 3 + 2 \cdot 1 + 3 \cdot 3 + 4 \cdot 1 + 1 \cdot 3 + 2 \cdot 1 + 3 \cdot 3 + 4 \cdot 1 + 5 \cdot 3 \\
 &= 5 + 27 + 0 + 3 + 2 + 9 + 4 + 3 + 2 + 9 + 4 + 15 \\
 &= 83
 \end{aligned}$$

$$\begin{aligned}
 83 \bmod 10 &= 3 \\
 10 - 3 &= 7
 \end{aligned}$$

Die Prüfwziffer ist also **7**, der vollständige EAN-13 lautet **5 9 0 1 2 3 4 1 2 3 4 5 7**.



Bleistift-
Übung 4

Echte Codes nachrechnen

- (a) Schnapp dir das Produkt vom Tisch, schreibe die 13 Ziffern auf, **decke die letzte Ziffer ab** und rechne sie aus. Stimmt sie mit der gedruckten überein?
- (b) Berechne die Prüfwziffer für diese 12 Datenziffern: 400638133393. (*Tipp: Es ist eine deutsche EAN; das 4 zeigt das Herkunftsland.*)
- (c) Hier sind drei EAN-13. Genau einer davon ist gefälscht – welcher?
 - 4006381333931
 - 9783446435001
 - 4012345678902

Der EAN-13-Validator

Python-
Einheit 2

```
def ean13_pruefziffer(zwoelf_ziffern):
    """Berechnet die EAN-13-Prüfziffer für eine Liste von 12 Ziffern."""
    summe = 0
    for i, d in enumerate(zwoelf_ziffern):
        gewicht = 1 if i % 2 == 0 else 3
        summe += d * gewicht
    return (10 - summe % 10) % 10

def ean13_ist_guelteig(ean):
    """ean ist ein String mit 13 Ziffern."""
    ziffern = [int(c) for c in ean]
    erwartet = ean13_pruefziffer(ziffern[:12])
    return erwartet == ziffern[12]

# Tests:
print(ean13_ist_guelteig("4006381333931")) # True
print(ean13_ist_guelteig("4006381333932")) # False (letzte Ziffer falsch)
```

Aufgabe 4.1 - Eigenes Produkt Tippe deinen echten EAN-13 in die Funktion ein. Wenn du False siehst, hast du dich verschrieben – schau nochmal hin.

Block 5 · Was EAN-13 *nicht* erkennt (≈ 30 min)

Das Spannende kommt jetzt: Jedes Verfahren hat **blinde Flecken**. Bei der Parität waren es Doppelfehler. Bei EAN-13 ist es subtiler.

Erste Vermutungen

- Behauptung: „EAN-13 erkennt jeden Tippfehler in einer einzelnen Ziffer.“ Stimmt das? Versuche, einen Gegenbeweis zu finden, indem du in 4006381333931 eine einzelne Ziffer veränderst und nachrechnest. Probiere mehrere.
- Tippfehler #2: **Zifferndreher** (zwei benachbarte Ziffern vertauscht). Aus 4006381333931 wird zum Beispiel 0406381333931. Rechne nach, ob die Prüfziffer das merkt.
- Probiere mehrere Zifferndreher aus. Findest du einen, den EAN-13 **nicht** erkennt?

Mit Brute Force die blinden Flecken finden

Computer machen sowas viel schneller als wir. Lass den Computer **alle möglichen Vertauschungen benachbarter Ziffern** systematisch testen:



Bleistift-
Übung 5



Python-
Einheit 3

```
def teste_alle_zifferndreher(ean):
    nicht_erkant = []
    ziffern = list(ean)
    for i in range(12):
        if ziffern[i] == ziffern[i+1]:
            continue # gleiche Ziffern → kein echter Dreher
        getauscht = ziffern.copy()
        getauscht[i], getauscht[i+1] = getauscht[i+1], getauscht[i]
        kandidat = ''.join(getauscht)
        if ean13_ist_gueltig(kandidat):
            nicht_erkant.append((i, kandidat))
    return nicht_erkant

print(teste_alle_zifferndreher("4006381333931"))
```

Aufgabe 5.1 Lass das Skript laufen. Welche Position(en) tauchen auf?

Aufgabe 5.2 - Das Muster finden Schau dir die Stellen, an denen ein Zifferndreher *nicht* erkannt wird, genau an. Was haben die beiden vertauschten Ziffern gemeinsam?

Hinweis: Es hat mit ihrer **Differenz** zu tun.

Aufgabe 5.3 - Wie viele Einzelfehler werden erkannt? Schreibe Code, der für einen gültigen EAN-13 alle möglichen Einzelfehler durchprobiert (jede der 13 Stellen kann zu jeder anderen der 9 Ziffern verändert werden – das sind $13 \times 9 = 117$ Möglichkeiten) und zählt, wie viele davon erkannt werden.

Block 6 · Reflexion und Ausblick (≈ 15 min)

Was wir heute gelernt haben

- Fehlerkorrektur kostet **Redundanz** – die Frage ist, wie clever man sie nutzt.
- **Modulo** ist das Universalwerkzeug, weil es Zahlen klein hält.
- **Parität** erkennt jeden Einzelfehler, aber keinen Doppelfehler. Sie kann nicht korrigieren.
- **EAN-13** erkennt jeden Einzelfehler, aber genau die Hälfte der Zifferndreher bleibt unentdeckt.
- Verfahren testet man am besten **systematisch mit Brute Force** – Computer sind dafür perfekt.

Drei Fragen zum Mitnehmen

1. Warum ist es eigentlich nicht trivial, einen Fehler nicht nur zu erkennen, sondern auch zu **korrigieren**?
2. Wenn EAN-13 Zifferndreher mit Differenz 5 nicht erkennt – könnte man das Verfahren so verbessern, dass es das doch tut? Was müsste man ändern?
3. Wie viele Datenziffern bräuchten wir wohl, um einen einzelnen Fehler nicht nur zu erkennen, sondern auch zu **korrigieren**?

Vorschau Tag 2

Morgen schauen wir uns weitere Prüfwziffer-Verfahren aus der Praxis an: die **ISBN-10** (mit Modulo 11 und einem Trick für 1-Ziffer-Korrektur), den **Luhn-Algorithmus** für Kreditkarten – und die zentrale Frage: **Wie viel Redundanz brauche ich, um einen Fehler nicht nur zu finden, sondern selbst zu reparieren?** Damit landen wir beim Begriff der **Hamming-Distanz** – und sind nur noch ein paar Tage von Reed-Solomon und Datamatrix entfernt.

Lösungen (erst nach eigenem Versuch ansehen!)

Bleistiftübung 1 - Wiederholungscode

1. Mehrheitsentscheid: 111→1, 010→0, 001→0, 110→1, 000→0, 101→1.
2. Pro Block korrigieren: 1 Bitfehler. Erkennen (ohne zu korrigieren): 2 Bitfehler. (3 Fehler = alle gekippt → wird falsch korrigiert!)
3. Redundanz: 2 von 3 Bits sind Redundanz, also etwa 67 %.
4. Viel zu teuer. In der Praxis braucht man Verfahren, die mit z. B. 10 % Redundanz auskommen.

Bleistiftübung 2 - Modulo

Aufgabe	Ergebnis
29 mod 7	1
100 mod 9	1
1000 mod 11	10
(47 + 38) mod 10	5
(8 · 7) mod 10	6
0 mod 13	0

Bleistiftübung 3 - Parität

Datenwort	P	Gesendet
1010101	0	10101010
1111000	0	11110000
0000001	1	00000011
1100110	0	11001100
0000000	0	00000000

(a)

(b) Anzahl der Einsen zählen, ungerade = fehlerhaft:

- 10110011 → 5 Einsen → **fehlerhaft**
- 11000001 → 3 Einsen → **fehlerhaft**
- 00000000 → 0 → ok
- 01010100 → 3 → **fehlerhaft**
- 11111111 → 8 → ok

(c) Bei zwei gekippten Bits kippt auch die Anzahl der Einsen zweimal um → die Parität bleibt **gleich** → Fehler **nicht erkannt**.

(d) Parität erkennt genau **eine ungerade Anzahl gekippter Bits** (1, 3, 5, ... Fehler). Gerade Fehleranzahlen bleiben unbemerkt.

Bleistiftübung 4 - EAN-13

(b) 400638133393:

$$\begin{aligned}
& 4 \cdot 1 + 0 \cdot 3 + 0 \cdot 1 + 6 \cdot 3 + 3 \cdot 1 + 8 \cdot 3 + 1 \cdot 1 + 3 \cdot 3 + 3 \cdot 1 + 3 \cdot 3 + 9 \cdot 1 + 3 \cdot 3 \\
& = 4 + 0 + 0 + 18 + 3 + 24 + 1 + 9 + 3 + 9 + 9 + 9 = 89 \\
& (10 - 89 \bmod 10) \bmod 10 = (10 - 9) \bmod 10 = 1
\end{aligned}$$

→ Prüfwziffer **1**, voller Code: 4006381333931.

(c) Mit dem Verfahren ausrechnen:

- 4006381333931 → gültig (Prüfwziffer 1 ☐)
- 9783446435001 → gültig (ISBN-13, Prüfwziffer 1 ☐)
- 4012345678902 → Summe = $4 \cdot 1 + 0 \cdot 3 + 1 \cdot 1 + 2 \cdot 3 + 3 \cdot 1 + 4 \cdot 3 + 5 \cdot 1 + 6 \cdot 3 + 7 \cdot 1 + 8 \cdot 3 + 9 \cdot 1 + 0 \cdot 3$
 $= 4 + 0 + 1 + 6 + 3 + 12 + 5 + 18 + 7 + 24 + 9 + 0 = \mathbf{89}$, Prüfwziffer also **1**, nicht 2 → **gefälscht**.

Aufgabe 3.1 - Lösung

```

import itertools

richtig = 0
gesamt = 0
for daten in itertools.product([0,1], repeat=7):
    daten = list(daten)
    p = paritaetsbit(daten)
    gesendet = daten + [p]
    for pos in range(8):
        empfangen = kippe_bit(gesendet, pos)
        gesamt += 1
        if not ist_guelteig(empfangen):
            richtig += 1
print(f"{richtig}/{gesamt} Einzelfehler erkannt")
# → 1024/1024 (100 %)

```

Aufgabe 3.2 - Lösung

```

import itertools

richtig, gesamt = 0, 0
for daten in itertools.product([0,1], repeat=7):
    daten = list(daten)
    gesendet = daten + [paritaetsbit(daten)]
    for p1, p2 in itertools.combinations(range(8), 2):
        empfangen = kippe_bit(kippe_bit(gesendet, p1), p2)
        gesamt += 1
        if not ist_guelteig(empfangen):
            richtig += 1
print(f"{richtig}/{gesamt} Doppelfehler erkannt")
# → 0/3584 (0 %) Parität erkennt KEINEN Doppelfehler.

```

Aufgabe 5.1 / 5.2 - Lösung

Bei 4006381333931 werden Vertauschungen an Position 1↔2 (00/06 – egal, weil eine Ziffer 0 ist) ... usw. Wenn man systematisch alle EANs ausprobiert, erkennt man:

Ein Zifferndreher zwischen benachbarten Ziffern a und b wird genau dann nicht erkannt, wenn $|a - b| = 5$.

Begründung: Die Differenz, die der Tausch in der Summe macht, ist $(b \cdot 1 + a \cdot 3) - (a \cdot 1 + b \cdot 3) = 2(a - b)$. Damit das modulo 10 verschwindet, muss $2(a-b) \equiv 0 \pmod{10}$, also $a - b \equiv 0 \pmod{5}$. Bei einer echten Vertauschung ($a \neq b$) heißt das: $|a-b| = 5$.

→ Von den 45 möglichen Ziffernpaaren mit $a \neq b$ haben 5 Paare die Differenz 5: (0,5), (1,6), (2,7), (3,8), (4,9). Das sind $5/45 \approx 11\%$ aller möglichen Zifferndreher, die EAN-13 **nicht** erkennt.

Aufgabe 5.3 - Lösung

```
def teste_einzelfehler(ean):
    erkannt = 0
    gesamt = 0
    ziffern = list(ean)
    for pos in range(13):
        original = ziffern[pos]
        for neu in '0123456789':
            if neu == original:
                continue
            kandidat = ziffern.copy()
            kandidat[pos] = neu
            gesamt += 1
            if not ean13_ist_gueltig(''.join(kandidat)):
                erkannt += 1
    return erkannt, gesamt

print(teste_einzelfehler("4006381333931"))
# → (117, 117) – alle 117 Einzelfehler werden erkannt.
```

Antworten zu den drei Reflexionsfragen

1. **Erkennen** heißt nur „etwas stimmt nicht“. **Korrigieren** heißt „ich weiß, was ursprünglich da stand“. Dafür müssen sich gültige Codewörter so stark unterscheiden, dass sich nach einem Fehler immer noch das ursprüngliche Codewort eindeutig „am nächsten“ anbietet → führt zur **Hamming-Distanz** (Tag 2/3).
2. Statt der Gewichte 1 und 3 könnte man andere Gewichte (z. B. mit Modulo 11) verwenden – das macht die ISBN-10 (kommt morgen).
3. Spoiler: Für 1-Bit-Korrektur reichen erstaunlich wenige Zusatzbits – siehe **Hamming-Code (7,4)** in Tag 3.